

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Sistemas Operativos II

PROYECTO FINAL

K-MONITOR

Monitor de Salud del Sistema en Tiempo Real mediante un Módulo Cargable del Kernel Linux

Catedrático: René Ornelis

Curso: Sistemas Operativos II

Integrantes:

- Lorenzo Antonio Pichilla Barillas 6590-19-12094
- Kimberly Rocío López Peraza 1390-23-6322
- Luis Fernando Alvarado Ramos 7590-20-466

Fecha de entrega: Junio 2026

ÍNDICE

1. Introducción
2. Objetivos 2.1 Objetivo General 2.2 Objetivos Específicos
3. Marco Teórico 3.1 Kernel Linux 3.2 Módulos Cargables del Kernel (LKM) 3.3 Sistema de Archivos Virtual /proc 3.4 Administración de Memoria 3.5 Administración de Procesos
4. Arquitectura de la Solución
5. Desarrollo e Implementación 5.1 Estructura del Proyecto 5.2 Funcionamiento General 5.3 Obtención de Información de Memoria 5.4 Obtención de Información de Procesos 5.5 Creación de la Entrada en /proc 5.6 Eliminación de Recursos
6. Estructuras de Datos del Kernel Utilizadas
7. Manual de Compilación y Ejecución
8. Resultados Obtenidos
9. Conclusiones

1. INTRODUCCIÓN

Los sistemas operativos modernos administran los recursos de hardware y software mediante un componente central denominado Kernel. Este componente es responsable de gestionar procesos, memoria, dispositivos y mecanismos de comunicación entre aplicaciones y hardware.

El presente proyecto consiste en el desarrollo de un Módulo Cargable del Kernel (Loadable Kernel Module - LKM) para Linux denominado K-Monitor. El módulo permite visualizar información relevante sobre el estado actual del sistema operativo, específicamente el uso de memoria RAM y los procesos activos del sistema.

La información recolectada es expuesta al espacio de usuario mediante una entrada dentro del sistema de archivos virtual /proc, permitiendo consultar los datos utilizando herramientas estándar de Linux.

2. OBJETIVOS

2.1 Objetivo General

Desarrollar un módulo cargable del kernel Linux capaz de monitorear recursos del sistema y presentar dicha información al usuario mediante una interfaz basada en el sistema de archivos virtual /proc.

2.2 Objetivos Específicos

- Comprender la estructura interna del kernel Linux.
 - Implementar un módulo cargable utilizando lenguaje C.
 - Obtener estadísticas de memoria directamente desde el kernel.
 - Recorrer la lista de procesos activos utilizando estructuras internas del sistema operativo.
 - Crear una interfaz de consulta accesible desde el espacio de usuario.
 - Aplicar buenas prácticas en el desarrollo de módulos del kernel.
-

3. MARCO TEÓRICO

3.1 Kernel Linux

El Kernel es el núcleo del sistema operativo Linux. Actúa como intermediario entre las aplicaciones y el hardware, administrando recursos como memoria, CPU, dispositivos de entrada y salida y procesos.

Su función principal es garantizar que múltiples programas puedan ejecutarse de forma eficiente y segura.

3.2 Módulos Cargables del Kernel (LKM)

Los Loadable Kernel Modules permiten extender las funcionalidades del kernel sin necesidad de recompilarlo.

Los módulos pueden cargarse dinámicamente utilizando:

`insmod`

y descargarse mediante:

`rmmmod`

Esta característica facilita el desarrollo y prueba de nuevas funcionalidades.

3.3 Sistema de Archivos Virtual /proc

El directorio /proc es un sistema de archivos virtual utilizado por Linux para exponer información interna del kernel.

No contiene archivos físicos almacenados en disco; su contenido se genera dinámicamente cuando el usuario realiza una lectura.

Ejemplos:

`/proc/cpuinfo`

`/proc/meminfo`

`/proc/version`

3.4 Administración de Memoria

La administración de memoria es responsabilidad del kernel y consiste en asignar, liberar y controlar el uso de memoria física y virtual.

Para este proyecto se utiliza la estructura `sysinfo`, la cual proporciona información sobre:

- Memoria total.
 - Memoria libre.
 - Tamaño de unidad de memoria.
-

3.5 Administración de Procesos

Un proceso representa una instancia en ejecución de un programa.

Linux mantiene una estructura denominada `task_struct` para cada proceso existente.

Esta estructura almacena:

- PID.
 - Nombre.
 - Estado.
 - Prioridad.
 - Información de planificación.
-

4. ARQUITECTURA DE LA SOLUCIÓN

La solución implementada sigue una arquitectura de tres niveles:

Nivel de Usuario ↓ Archivo Virtual /proc/kmonitor_grupo1 ↓ Módulo K-Monitor ↓ Kernel Linux

El usuario interactúa con el sistema mediante el comando:

```
cat /proc/kmonitor_grupo1
```

El archivo virtual solicita la información al módulo.

El módulo consulta las estructuras internas del kernel y devuelve los resultados formateados.

5. DESARROLLO E IMPLEMENTACIÓN

5.1 Estructura del Proyecto

kmonitor/

├─ kmonitor.c

├─ Makefile

└─ Documentacion.pdf

5.2 Funcionamiento General

Cuando el módulo es cargado:

1. Se registra el módulo.
2. Se crea la entrada en /proc.
3. Se habilita la lectura de información.

Cuando el usuario consulta el archivo:

1. Se obtiene información de memoria.
2. Se recorren los procesos activos.
3. Se genera la salida utilizando seq_file.

Cuando el módulo es descargado:

1. Se elimina la entrada en /proc.
 2. Se liberan los recursos utilizados.
-

5.3 Obtención de Información de Memoria

La función `si_meminfo()` llena una estructura `sysinfo` con los datos actuales de memoria del sistema.

Posteriormente se calculan:

- Memoria total.
 - Memoria libre.
 - Memoria utilizada.
 - Porcentaje de uso.
-

5.4 Obtención de Información de Procesos

La macro `for_each_process()` permite recorrer todos los procesos activos.

Para cada proceso se obtiene:

- PID.
- Nombre.
- Estado.

Estos datos son mostrados al usuario mediante `seq_printf()`.

5.5 Creación de la Entrada en /proc

La función `proc_create()` crea dinámicamente el archivo:

`/proc/kmonitor`

Esta entrada actúa como interfaz entre el usuario y el módulo.

5.6 Eliminación de Recursos

La función `proc_remove()` elimina correctamente la entrada creada cuando el módulo es descargado.

Esto evita referencias inválidas dentro del sistema de archivos virtual.

6. ESTRUCTURAS DE DATOS DEL KERNEL UTILIZADAS

`task_struct`

Representa un proceso dentro del kernel Linux.

Campos utilizados:

- `pid`
 - `comm`
 - `__state`
-

`for_each_process()`

Macro utilizada para recorrer la lista de procesos activos.

Permite acceder a cada `task_struct` existente.

`sysinfo`

Estructura utilizada para obtener estadísticas globales de memoria.

Campos utilizados:

- `totalram`
 - `freeram`
 - `mem_unit`
-

`proc_ops`

Define las operaciones asociadas a la entrada creada en `/proc`.

Operaciones utilizadas:

- `proc_open`
 - `proc_read`
 - `proc_lseek`
 - `proc_release`
-

seq_file

Interfaz utilizada para generar salidas extensas de forma segura y eficiente.

Permite imprimir grandes cantidades de información sin problemas de memoria.

7. MANUAL DE COMPILACIÓN Y EJECUCIÓN

Compilación:

`make`

Carga del módulo:

`sudo insmod kmonitor.ko`

Lectura de información:

`cat /proc/kmonitor`

Verificación:

`ls /proc | grep kmonitor`

Descarga del módulo:

`sudo rmmod kmonitor`

8. RESULTADOS OBTENIDOS

El módulo desarrollado cumplió satisfactoriamente con los requerimientos establecidos.

Se logró:

- Crear una entrada funcional dentro de `/proc`.

- Obtener estadísticas de memoria desde el kernel.
- Listar procesos activos.
- Mostrar PID, nombre y estado.
- Eliminar correctamente la entrada al descargar el módulo.

No se presentaron errores críticos ni Kernel Panic durante las pruebas realizadas.

9. CONCLUSIONES

1. El desarrollo del proyecto permitió comprender el funcionamiento interno del kernel Linux y su interacción con los recursos del sistema.
 2. El uso de módulos cargables facilita la extensión de funcionalidades sin necesidad de modificar o recompilar el kernel completo.
 3. La estructura `task_struct` constituye uno de los elementos fundamentales para la administración de procesos dentro de Linux.
 4. El sistema de archivos virtual `/proc` proporciona un mecanismo eficiente para exponer información interna del kernel hacia el espacio de usuario.
 5. La implementación de K-Monitor permitió aplicar conocimientos de memoria, procesos y estructuras internas del sistema operativo en un entorno real.
 6. El proyecto fortaleció las competencias relacionadas con programación de bajo nivel y administración de sistemas Linux.
-